PYTHON PROGRAMMING LAB

Lab: Building a Student Score Recorder with File Handling

1. Background

Imagine you are developing a program for a teacher who needs to record students' names and scores.

The teacher should be able to add new students, view previously recorded scores, and calculate the average score of the class.

The information must remain available even after the program is closed.

You will develop a **Student Score Recorder** that uses a file to store records permanently.

Each student will be represented by a dictionary:

```
student = {
   "name": "Ama",
   "score": 85
}
```

Multiple student records will be stored in a list:

```
students = [
   {"name": "Ama", "score": 85},
   {"name": "Kojo", "score": 72}
]
```

The program will save this list to a file and load it when it starts.

3. Lab instructions

Follow these instructions throughout the activity.

1. Create a folder named python_file_lab.
2. Inside the folder, create a Python file named score_recorder.py.
3. Copy the starter code provided below into the file.
4. Read each comment beginning with TODO. These comments indicate where you must complete the code.
5. Complete one function at a time. Run the program and test your changes before moving to the next function.

6. Do not remove the existing code unless you understand why a change is necessary.
7. When testing, at least three students enter with different scores.
8. Close the program and run it again to verify that the records are still available.

Important: Keep score_recorder.py and students.txt in the same folder for this lab.

4. Starter code

Complete the missing sections marked with TODO. The program structure is provided to help you focus on the concepts being practiced.

```
FILENAME = "students.txt"
# TASK 1: Load saved student records
def load_students():
    try:
        with open(FILENAME, "r") as file:
            #open the file

        return students

    except FileNotFoundError:
        # TODO 1:
        # If the file does not exist, return an empty list.
        pass


# TASK 2: Save student records
def save_students(students):
    with open(FILENAME, "w") as file:
        # TODO 2:
        # save the students list.
        pass


# TASK 3: Add a new student
def add_student(students):
    name = input("Enter student name: ").strip()

    if name == "":
```

```
        print("Name cannot be empty.")
        return

    try:
        score = float(input("Enter student score: "))

        if score < 0 or score > 100:
            print("Score must be between 0 and 100.")
            return

    except ValueError:
        print("Please enter a valid number.")
        return

    # TODO 3:
    # Create a dictionary with the student's name and score.
    # Add the dictionary to the students list.
    # Call save_students() to save the updated list.
    # Print a success message.


# TASK 4: Display all saved students
def view_students(students):
    if len(students) == 0:
        print("No student records found.")
        return

    print("\n--- STUDENT RECORDS ---")

    # TODO 4:
    # Use a for loop to visit every student in the list.
    # Print each student's name and score.
    # Remember that each student is a dictionary.


# TASK 5: Calculate the class average
def show_average(students):
    if len(students) == 0:
        print("No scores available.")
```

```
        return

    total = 0

    # TODO 5:
    # Use a for loop to add each student's score to total.

    # TODO 6:
    # Calculate the average by dividing total
    # by the number of students.
    # Print the average rounded to 2 decimal places.


# TASK 6: Create the main menu
def main():
    students = load_students()

    while True:
        print("\n--- STUDENT SCORE RECORDER ---")
        print("1. Add student")
        print("2. View student records")
        print("3. Calculate class average")
        print("4. Exit")

        choice = input("Choose an option: ")

        # TODO 7:
        # If choice is "1", call add_student(students).
        # If choice is "2", call view_students(students).
        # If choice is "3", call show_average(students).
        # If choice is "4", print a goodbye message and break.
        # Otherwise, print an invalid option message.


if __name__ == "__main__":
    main()
```